Indic Prompt Compressor

Nov 4, 2026 · @Sarthak

Overview

A layer that makes AI cheaper and faster for people who use it in Indian languages. Text in scripts like Devanagari, Kannada or Tamil usually costs an AI model far more "tokens" than the same text in English. The prompt compressor shrinks Indian-language input before it reaches the model, while keeping the meaning and the quality of the answer intact.

The problem

AI language models read and write text in small pieces called tokens, and they charge, slow down and run out of space based on how many tokens they handle. Most of these models were designed mainly around English. As a result, the same sentence in an Indian language often breaks into several times more tokens than its English version.

This creates a quiet disadvantage for Indian-language users:

- **Higher cost:** the same question or document costs more to process in Hindi or Tamil than in English.
- **Slower responses:** more tokens means longer waits.
- **Less room:** the model can fit less Indian-language text at once, so long documents or conversations get cut short sooner.
- **An unfair gap:** products that serve Indian languages pay more per user, which can push them to offer less or charge more.

Who it is for

- **Companies building Indian-language AI products,** such as chatbots, voice assistants and customer-support tools, that want to cut their AI costs.
- **Startups and student developers** with small budgets who want to build in Indian languages.
- **Government and public-service platforms** that serve millions of citizens in their own languages and need to keep costs low at scale.
- **Researchers** studying fairness and efficiency of AI across languages.

The idea

The compressor sits between an application and the AI model. Every Indian-language prompt passes through it first and comes out shorter, so the model processes fewer tokens for the same request.

The idea has two parts:

1. **Measure the gap.** Show clearly how many more tokens each Indian language costs compared with English, across different AI models. This makes the problem visible and gives a baseline.
2. **Close the gap.** Apply ways of shortening the prompt, and for each request choose the one that saves the most while keeping answer quality above an agreed level. Each result reports how much was saved and how much quality, if any, was lost.

Solutions for the initial phase

These are the approaches we have identified to explore first. Part of the project is finding out which works best for which language and task.

- **Removing redundancy:** cut filler words, repeated context and boilerplate that add tokens but no meaning.
- **Keeping only what matters:** identify the words that carry the meaning of the prompt and drop the less important ones.
- **Going through English:** translate the prompt to English, get the answer, and translate it back. This is often cheaper, but some nuance may be lost.
- **Changing the script:** write the same Indian-language text in Roman letters, which some models handle with fewer tokens, at a possible cost to quality.
- **Choosing automatically:** for each request, pick the cheapest approach that still keeps the answer good enough.
- **Showing the savings:** for every prompt, report tokens saved, cost saved and any change in answer quality, so users can see the trade-off.

What makes it different

- **It targets a problem few people talk about.** The extra cost of Indian languages in AI is real but rarely measured or addressed directly.

- **It is honest about trade-offs.** Instead of only claiming savings, it shows exactly how much quality is kept for each saving.
- **It works with any AI product.** As a layer in front of the model, it can help many Indian-language applications without changing how they work.
- **It turns fairness into numbers.** Measuring the cost gap across languages makes the case for better support of Indian languages concrete.

First-phase scope and open questions

The first phase aims to prove the idea on a small, focused set of cases.

- **Two or three languages:** for example Hindi, Kannada and Tamil.
- **One or two everyday tasks:** such as answering questions and summarising text.
- **A clear result:** how much each approach saves, and how much answer quality it keeps.

Open questions:

- [ ] Which languages and tasks will make the strongest case?
- [ ] How much quality loss is acceptable in exchange for savings?
- [ ] How will we judge whether an answer is still "good enough" after compression?
- [ ] If going through English turns out to beat everything else, how do we present that finding?